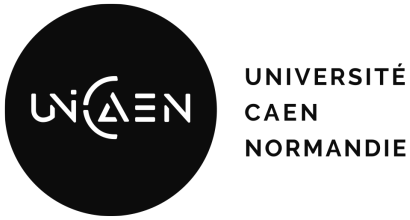
Suivi des connaissances - Projet Multi-Agents

Tom David, Pasquier Titouan, Guillais Marius, Chepy Maxim

Semestre 2 - 2025/2026



Sommaire

1 Description générale **3**
- 1.1 Système multi-agents . . . 3
- 1.2 Objectifs du projet . . . 3
- 1.3 Organisation des fichiers . . . 3

2 Agents **4**
- 2.1 Fonctionnement . . . 4
- 2.2 Implémentation . . . 4
- 2.3 Belief state . . . 5

3 Implémentations **6**
- 3.1 Hanabi . . . 6
  - 3.1.1 Principe . . . 6
  - 3.1.2 Implémentation . . . 6
- 3.2 Jeu de la ville . . . 6
  - 3.2.1 Principe . . . 6
  - 3.2.2 Implémentation . . . 7
  - 3.2.3 Politique PCC et raisonnement du premier ordre . . . 7
  - 3.2.4 Raisonnement du second ordre . . . 8

4 Observations **9**
- 4.1 Core . . . 9
- 4.2 Jeu de la ville . . . 9
- 4.3 Hanabi . . . 9
- 4.4 Belief state et filtrage . . . 10

5 Conclusion **11**
- 5.1 Avancée actuelle . . . 11
- 5.2 Répartition des tâches . . . 11
- 5.3 Perspectives . . . 11

1 Description générale

1.1 Système multi-agents

Un système multi-agents est un ensemble d'agents autonomes agissant dans un environnement partagé et interagissant selon certaines règles.

Ce type d'implémentation a pour objectif de décentraliser la résolution de problèmes complexes. Cette approche s'inspire notamment du comportement des fourmis et des abeilles : l'objectif global est décomposé en tâches simples, réalisées par des agents autonomes qui ne se coordonnent pas nécessairement de façon explicite.

On retrouve cette méthode dans de nombreux domaines : tri du couvain, exploration et couverture après des catastrophes naturelles, dilemme du prisonnier itéré, etc.

1.2 Objectifs du projet

Ce projet porte sur les systèmes multi-agents dans lesquels les agents ne peuvent pas communiquer directement entre eux. Chaque agent ne peut se baser que sur ses propres observations de l'environnement et sur sa connaissance de la stratégie des autres agents.

L'objectif est de concevoir un modèle générique d'agent, robuste et adaptable à n'importe quel scénario respectant certaines contraintes, implémenté en Java.

Pour illustrer cette démarche, nous nous sommes appuyés sur deux scénarios : le **jeu de la ville** (problème du rendez-vous) et **Hanabi**.

1.3 Organisation des fichiers

Le package `core` contient l'ensemble des classes génériques constituant le framework multi-agents.

Les packages `hanabi` et `ville` contiennent les implémentations spécifiques à chaque scénario.

Pour lancer le projet, deux méthodes sont disponibles :
- Les scripts shell `run_ville2.sh` et `run_hanabi.sh` exécutent respectivement le jeu de la ville et Hanabi.
- Les commandes Maven `mvn exec:java -Dexec.mainClass="ville.MainVille"` et `mvn exec:java -Dexec.mainClass="hanabi.MainHanabi"` permettent également de lancer les simulations.

2 Agents

2.1 Fonctionnement

Un agent doit être capable de s'adapter à n'importe quel environnement et de choisir une action à chaque pas de temps, en se basant sur ses observations passées et ses actions précédentes.

De plus, il doit pouvoir être réutilisé dans des implémentations très diverses sans modification et sans perte d'efficacité.

2.2 Implémentation

Pour répondre à ces critères, nous avons cherché une modélisation générale permettant de représenter n'importe quel environnement. Nous avons rapidement constaté que cela était impossible de façon universelle, et avons donc opté pour une modélisation fournie au cas par cas, propre à chaque scénario.

La même réflexion s'applique à la **politique** : la stratégie de l'agent dépend du contexte et doit être fournie par le scénario.

Sur le plan technique, la classe **Agent** prend en paramètre un identifiant, une **politique** et un **historique**. La politique est une interface dont l'implémentation concrète est fournie par chaque scénario. L'historique est une liste de paires (action, observation) accumulées au fil de la simulation.

Une **politique par défaut**, `PolitiqueRandom`, effectue un choix aléatoire à chaque pas de temps.

Nous avons également implémenté une classe **ActionJointe**, qui associe à chaque agent l'action qu'il a réalisée à un pas de temps donné. Le **simulateur** prend en entrée un jeu et la liste des agents, et orchestre la boucle de simulation : collecte des actions via les politiques, application au jeu, génération des observations, et mise à jour des historiques.

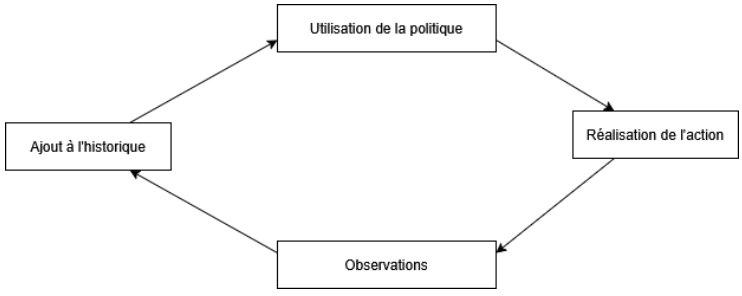


Figure 1 – Cycle du simulateur

2.3 Belief state

Une des principales avancées du second semestre est l'introduction du **belief state** (état de croyance). Un agent n'observe pas directement l'état global du jeu ; il accumule des observations partielles. Le belief state représente l'ensemble des états compatibles avec toutes les observations reçues jusqu'au présent.

La classe `BeliefState<E>` maintient une liste d'états compatibles et la met à jour à chaque pas de temps grâce à un `BeliefStateFilter<E>`.

Le filtre génère toutes les transitions possibles depuis chaque état compatible (en fixant l'action propre de l'agent observateur et en énumérant toutes les combinaisons d'actions pour les autres agents), puis ne conserve que les états successeurs dont l'observation simulée correspond à l'observation réelle reçue.

La méthode abstraite `simulerObservation` a donc été ajoutée à la classe `Jeu` pour permettre ce calcul.

3 Implémentations

3.1 Hanabi

3.1.1 Principe

Hanabi est un jeu de société coopératif dont l'objectif est, pour chacune des 5 couleurs, de placer les cartes dans l'ordre de 1 à 5.

La particularité du jeu est que chaque joueur ne voit pas ses propres cartes, mais voit celles des autres joueurs. À chaque tour, un joueur peut jouer une de ses cartes, utiliser un jeton d'indice pour informer un autre joueur de la composition de sa main, ou défausser une carte pour récupérer un jeton d'indice.

Si un joueur pose une carte incorrecte, l'équipe reçoit un jeton d'erreur. Le jeu s'arrête à 3 erreurs (défaite, 0 point) ou quand la pioche est vide (victoire partielle ou totale). Le score maximal est de 25 points.

Pour être efficaces, les joueurs doivent déduire des informations non explicitement communiquées, et tenir compte de ce que savent les autres pour éviter les indices inutiles.

3.1.2 Implémentation

Chaque carte est représentée par un numéro et une couleur. Les mains des joueurs sont stockées dans une `Map<Agent, List<Carte»`, la pioche dans une `List<Carte>`, et les piles de jeu dans une `Map<String, Deque<Carte»` associant chaque couleur à une pile.

La boucle de jeu et les conditions de victoire sont implémentées dans `JeuHanabi`.
Les actions disponibles sont : `ActionPoser`, `ActionDefausse`, `ActionIndiquerCouleur`, `ActionIndiquerValeur` et `ActionVide` (pour l'agent inactif).

3.2 Jeu de la ville

3.2.1 Principe

Le jeu de la ville est une simulation dans laquelle plusieurs agents doivent se retrouver en un lieu de rendez-vous, sans pouvoir communiquer directement entre eux.

À chaque tour, un agent peut **se déplacer** vers un lieu voisin, **rester sur place**, ou **laisser un message** sur sa case actuelle. Certaines routes peuvent être fermées, de façon permanente ou temporaire, ce qui contraint les déplacements.

Pour se retrouver efficacement, les agents doivent raisonner sur la position

probable des autres agents et sur les informations qu'ils détiennent, afin d'agir en conséquence.

3.2.2 Implémentation

La ville est modélisée par un graphe non-orienté dont les sommets représentent des lieux (métro, centre-ville, etc.) et les arêtes des chemins possibles. La classe `Ville` gère l'ouverture et la fermeture des lieux.

L'**état du jeu** (`EtatRDV`) stocke trois informations :
— la position courante de chaque agent,
— les messages déposés sur chaque lieu,
— les routes fermées connues par chaque agent (`routesFermeesConnues`), ce qui permet au `BeliefState` de raisonner sur ce que chaque agent sait.

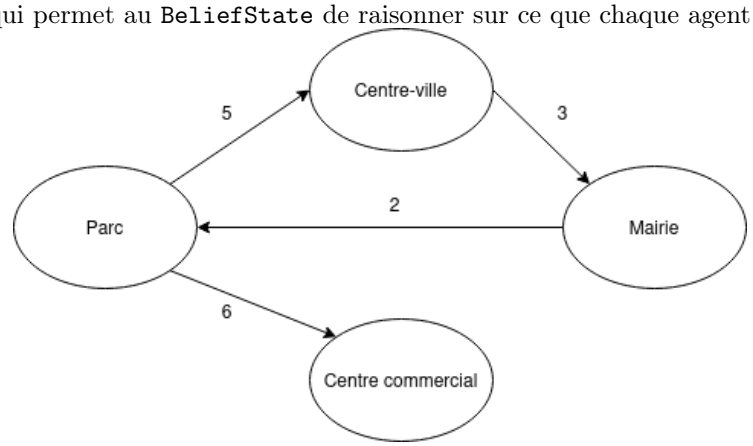


FIGURE 2 – Exemple de graphe du jeu de la ville

3.2.3 Politique PCC et raisonnement du premier ordre

La politique `PolitiquePCC` calcule le plus court chemin vers le lieu de rendez-vous par un algorithme de parcours en largeur (BFS) sur la carte connue de l'agent. Lorsqu'une route se révèle fermée (l'agent tente de se déplacer mais reste sur place), la politique met à jour sa carte et recalcule le chemin.

Au-delà du déplacement, la politique intègre un **raisonnement du premier ordre** : si un agent découvre une route fermée qu'il a lui-même tenté d'emprunter, il peut déposer un message de type `MessageRouteFerme` sur un lieu stratégique pour en informer les autres agents.

Le lieu de dépôt est calculé intelligemment via `calculerMeilleurDepot` : la politique cherche l'intersection entre le chemin probable d'un autre agent (estimé via le belief state) et le chemin de l'agent courant, afin de placer le message là où il a le plus de chances d'être lu.

Un type de message supplémentaire a été ajouté : `MessageAgentBloque`. Lorsqu'un agent se retrouve sur le même lieu qu'un autre agent qui vient de tenter de se déplacer sans y parvenir, l'observation enregistre cet événement. Cela permet à un agent témoin d'apprendre indirectement qu'une route est fermée sans l'avoir tentée lui-même.

3.2.4 Raisonnement du second ordre

Le **raisonnement du second ordre** consiste pour un agent à tenir compte non seulement de ce qu'il sait, mais aussi de ce que les autres agents savent déjà.

Concrètement, avant de déposer un message sur une route fermée, la politique vérifie via le belief state si au moins un autre agent a besoin de cette information, c'est-à-dire si son chemin probable passe par la route fermée *et* s'il ne la connaît pas encore (`routesFermeesConnues` de l'état).

Si tous les autres agents sont déjà informés, le message n'est pas déposé, évitant ainsi des actions inutiles.

4 Observations

Nous avons défini plus haut ce qu'est une politique. Cette dernière régit les choix des agents, mais ceux-ci n'ont pas accès directement à l'état global du jeu : par exemple, dans Hanabi un agent ne connaît pas sa propre main, et dans le jeu de la ville il ne sait pas si une route est fermée.

L'objectif est donc de maximiser les connaissances des agents via les observations, afin que la politique performe le mieux possible. Par exemple, si un agent tente de se déplacer vers un lieu X et se retrouve toujours à sa position initiale après le pas de temps, l'observation peut en déduire avec certitude que la route était fermée. La politique peut alors utiliser cette information pour les choix futurs.

4.1 Core

Dans le package `core`, la classe `Observation` est minimaliste : elle comporte un nom et une redéfinition de `equals`, ce qui permet de comparer une observation simulée et une observation réelle (utile pour le belief state).

Chaque implémentation fournit sa propre sous-classe d'observation. Le principe commun est que la collecte d'informations se fait dans le constructeur, avec pour arguments l'agent, l'état et l'action jointe.

4.2 Jeu de la ville

L'observation du jeu de la ville (`ObservationVille`) contient une `List<Message>`. Cette structure est adaptée car plusieurs informations de nature différente peuvent être obtenues en un seul pas de temps.

Trois types de messages sont actuellement définis :
- `MessageAgentPresent` : un autre agent se trouve sur le même lieu,
- `MessageRouteFerme` : une route est fermée (appris directement ou via un message déposé),
- `MessageRouteHoraire` : une route est fermée selon un horaire,
- `MessageAgentBloque` : un agent colocataire a tenté de quitter le lieu sans y parvenir, révélant indirectement qu'une route est fermée.

Les agents peuvent également déposer des messages de type `MessageRouteFerme` sur des lieux, que les autres agents liront lors de leur passage. Ce mécanisme de communication indirecte est le seul moyen de partager de l'information entre agents.

4.3 Hanabi

L'observation de Hanabi (`ObservationHanabi`) contient trois informations :
- la main des autres agents : `Map<Agent, List<Carte»`,

— l'état des piles de jeu : `Map<String, Deque<Carte»`,
— le nombre d'erreurs : un entier.

La politique utilise ces informations pour décider quelle action jouer.

4.4 Belief state et filtrage

Pour les scénarios où l'agent doit raisonner sur des états qu'il n'observe pas directement (comme la position des autres agents dans le jeu de la ville), le `BeliefState` maintient la liste des états compatibles avec toutes les observations reçues.

À chaque pas de temps, le `BeliefStateFilter` :

1. génère toutes les `Transition<E>` possibles depuis chaque état compatible, en fixant l'action propre de l'agent et en énumérant les combinaisons d'actions des autres agents,
2. simule l'observation que l'agent aurait reçue dans chaque transition via `simulerObservation`,
3. ne conserve que les états successeurs dont l'observation simulée correspond à l'observation réelle.

Ce mécanisme permet à la politique d'estimer les positions probables des autres agents, et ainsi de prendre des décisions plus informées (dépôt de messages, calcul du meilleur itinéraire).

5 Conclusion

5.1 Avancée actuelle

Nous disposons désormais d'un framework multi-agents fonctionnel, modulaire et adaptable. Le système générique (package `core`) gère la boucle de simulation, l'historique, le belief state et son filtrage.

Le **jeu de la ville** est entièrement fonctionnel : le BFS calcule le plus court chemin, les routes fermées sont détectées et propagées via les messages, et la politique intègre un raisonnement du premier et du second ordre. Le belief state permet à chaque agent d'estimer les positions probables des autres, ce qui améliore les décisions de dépôt de messages.

Le jeu **Hanabi** dispose d'une boucle de jeu complète avec toutes les actions (poser, défausser, indiquer couleur/valeur). Une politique simple est en place.

5.2 Répartition des tâches

Tom David a implémenté le framework générique (core), le jeu de la ville ainsi que le belief state et son mécanisme de filtrage. Au second semestre, il a développé la politique PCC avec le raisonnement du premier et du second ordre.

Titouan Pasquier a implémenté Hanabi et sa politique. Au second semestre, il a travaillé sur l'amélioration de la politique Hanabi.

Marius Guillais a travaillé sur le système de messages et d'observations dans le jeu de la ville, et a contribué à l'implémentation du belief state appliqué à ce scénario.

Maxim Chepy a contribué à l'exploration de politiques plus complexes et à la rédaction du rapport.

5.3 Perspectives

Plus généralement, le framework est prêt à accueillir de nouveaux scénarios. Il suffit d'implémenter les classes abstraites de `core` (Jeu, Etat, Observation, Action, Politique) et de fournir un `simulerObservation` pour bénéficier automatiquement du belief state.